> REAL-TIME VULNERABILITY SCANNER

Project README · Technical Documentation

Python 3.x	tkinter	pip-audit	threading	subprocess

■ 1. Project Overview

This desktop application performs a live security audit of all Python packages installed in the current environment. It integrates directly with **pip-audit** — a command-line tool that queries the PyPI and OSV.dev vulnerability advisory databases — and presents the results in a sleek, terminal-inspired GUI built with Python's built-in **tkinter** library.

The tool is designed for students and developers who want a quick visual overview of dependency vulnerabilities without using a browser or the command line.

■ 2. Technologies & Libraries Used

Component	Technology / Library	Purpose
GUI Framework	tkinter	Native Python desktop GUI toolkit
Scan Engine	pip-audit	Audits installed packages vs CVE databases
Vuln Database	PyPI / OSV.dev	Live online vulnerability advisory feeds
Concurrency	threading	Parallel animation + scan without UI freeze
System Call	subprocess	Spawns pip-audit as a child process
Alert Dialogs	messagebox	Error pop-ups (from tkinter.messagebox)

■ 3. File & Module Structure

The entire project is contained in a single Python file. Internally it is organized into three logical layers:

◆ 3.1 Theme / Constants Layer (top of file)

Six color constants are defined using hex strings to enforce the matrix-green-on-black visual identity throughout every widget.

```
BG_COLOR     = '#000000'   # Window background
FRAME_COLOR  = '#0D0D0D'   # Inner frame / Listbox bg
ACCENT_COLOR = '#00FF41'   # Matrix Green — text & borders
WARNING_COLOR= '#FF0000'   # Red — vulnerability entries
SUCCESS_COLOR= '#00FF41'   # Green — clean scan result
BUTTON_PRIMARY='#003B00'   # Dark green — button fill
```

◆ 3.2 Logic Layer (three functions)

All business logic lives in three functions called from the GUI layer.

Function	Description
animate_loading(stop_event)	Runs in a daemon thread. Animates a filling loading bar (■→■) inside status_label every 0.3 s until stop_event fires.

scan_thread_logic(stop_event)	Background thread. Calls pip-audit via subprocess, parses stdout, updates the Listbox and status_label with results.
start_audit_process()	Entry point triggered by the button. Creates a threading.Event, then launches both threads in parallel.

◆ 3.3 GUI Layer (tkinter widgets)

Widgets are stacked top-to-bottom using the pack() geometry manager. No external styling frameworks are used — all styling is done via tkinter widget options.

■ 4. Execution Flow

Step	Action	Detail
1	User clicks button	start_audit_process() is called
2	Threads launched	Animation thread + Scan thread start together
3	Animation runs	Loading bar animates in status_label (every 0.3s)
4	pip-audit executes	subprocess.run(['pip-audit','--format','columns'])
5	Output parsed	Lines checked for 'No known vulnerabilities'
6	stop_event.set()	Animation thread stops automatically
7	Results displayed	vuln_list Listbox populated; status_label updated

---

■ 5. Key Concepts Explained

◆ 5.1 threading.Event — stop_event

A threading.Event object is shared between the animation thread and the scan thread. When the scan completes (or errors), scan_thread_logic() calls stop_event.set(). The animate_loading() loop checks stop_event.is_set() on every iteration and exits cleanly — no thread is forcibly killed.

◆ 5.2 daemon=True Threads

Both threads are started with daemon=True. This means if the user closes the main window mid-scan, Python will not keep the process alive waiting for the threads to finish.

◆ 5.3 subprocess.run() with capture_output

```
result = subprocess.run(
    ['pip-audit', '--format', 'columns'],
    capture_output=True,
    text=True,
    timeout=120
)
```

capture_output=True redirects stdout and stderr into result.stdout / result.stderr. text=True decodes bytes to a Python string. timeout=120 prevents the scan from hanging indefinitely on slow networks.

◆ 5.4 Listbox Population

vuln_list.delete(0, tk.END) clears any previous scan results before inserting new ones. Each vulnerable package is prefixed with a red-circle emoji and inserted as a separate entry so the user can scroll through individual CVE entries.

■ 6. GUI Layout Overview

All widgets use pack() with top-to-bottom stacking. Key layout choices:

- Window size: 1120 × 780 px, fixed.
- Header: two Label widgets — project title (28 pt bold) and subtitle (14 pt).
- Button: flat relief, hand2 cursor, highlighted border in ACCENT_COLOR.
- Status Label: single line updated in real-time by animate_loading() and scan_thread_logic().
- Listbox: 18 rows visible, WARNING_COLOR (red) text, ACCENT_COLOR selection highlight.
- Architecture Frame: FRAME_COLOR background, shows engine/database/logic description.
- Footer: two Label widgets — project badge and developer credit.

■ 7. Installation & Usage

◆ Requirements

- Python 3.7 or higher (tkinter is included in the standard library)
- pip-audit package (not in stdlib — must be installed separately)

◆ Install pip-audit

```
pip install pip-audit
```

◆ Run the Application

```
python vuln_scanner.py
```

◆ Usage Steps

1. Launch the script — the dark GUI window opens.
2. Click [RUN LIVE SYSTEM AUDIT].
3. Watch the animated loading bar while pip-audit queries the CVE databases.
4. Results appear in the LIVE_AUDIT_LOG panel:

- Green message → no vulnerabilities found in your environment.
- Red entries → each line shows a vulnerable package name + CVE ID + fix version.

■ 8. Error Handling

Two exception types are caught inside scan_thread_logic():

- FileNotFoundError — raised when pip-audit is not installed. A messagebox pop-up instructs the user to run pip install pip-audit.
- Exception (catch-all) — any other runtime error (network timeout, permission denied, etc.) is shown in a generic error dialog so the app never silently crashes.

■ 9. Developer Information

Developer : Mahad Ali Magsi

Student ID : B2533060

Section : BSAI 3rd "A"

[END OF DOCUMENTATION]

Generated automatically from source code analysis.